PROJECT-BASED LEARNING (PBL) REPORT

DATABASE MANAGEMENT SYSTEMS

Small Database Application Development using Python



PROJECT TITLE

ALUMNI MENTORING AND MANAGEMENT SYSTEM

Submitted by

Nayan Mittal

Roll Number: 25WU0102182

Faculty: C Nagaraju

Programme: B.Tech CSE (Artificial Intelligence and Machine Learning)

Section: AIML Leopards

School of Technology | Woxsen University

Academic Year: 2026–27

Date of Submission: 12/10/2026

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing me with the opportunity to undertake this Project-Based Learning activity as part of the Database Management Systems course.

I thank the course faculty for the guidance and suggestions that supported my understanding of entity-relationship modelling, relational schema design, normalization, SQL constraints, database operations and application development. This project helped me connect database concepts with a practical alumni engagement scenario.

I also acknowledge the documentation and development tools used to prepare and demonstrate the prototype. The report distinguishes the current browser-based simulation from the relational database implementation proposed in the logical design.

UNDERTAKING

I hereby declare that the project entitled “Alumni Mentoring & Engagement Management System,” submitted as part of the Project-Based Learning activity for the Database Management Systems course, is my academic work carried out under the guidance of the course faculty.

The report describes the database design and the supplied application prototype. The prototype’s current data storage and functionality are reported as observed; database connectivity or operations that are not present in the supplied simulation are not claimed as implemented. Sources and tools referred to in preparing the report are acknowledged where applicable.

I understand that I am responsible for reviewing the report, verifying the implementation, and completing the submission details before submission.

Signature: _____

Date: 12/10/2026

Place: Woxsen University

Name:NayanMittal

RollNumber: 25WU0102182

Program: B.TechAI-ML

TABLE OF CONTENTS

Abstract

Keywords

List of Tables

List of Figures

List of Acronyms

1. Problem Statement, Solution Strategy and ER Diagram

1.1 Problem Statement

1.2 Objectives

1.3 Scope and Assumptions

1.4 Proposed Solution Strategy

1.5 Functional Requirements

1.6 Entity-Relationship Diagram (ERD)

1.7 Entity and Relationship Summary

2. Logical Database Design

2.1 Relational Schema

2.2 Keys and Integrity Constraints

2.3 Functional Dependencies

2.4 Normalization up to Third Normal Form (3NF)

2.5 Data Dictionary

2.6 Implementation of Logical Design: DDL and Sample Data

3. Application Development Using Logical Database Design

3.1 Application Overview

3.2 Development Environment and Database Connectivity

3.3 Application Modules and Database Mapping

3.4 CRUD Operations

3.5 Business Rules, Validation and Error Handling

3.6 Reports and Database Operations

3.7 Application Screens

3.8 Testing and Results

4. Conclusion

4.1 Limitations and Future Enhancements

5. References

6. Appendices

LIST OF TABLES

Table 1. Scope and Assumptions

Table 2. Proposed Solution Strategy

Table 3. Functional Requirements

Table 4. Entity and Relationship Summary

Table 5. Relational Schema

Table 6. Keys and Integrity Constraints

Table 7. Normalization Status up to 3NF

Table 8. Data Dictionary

Table 9. Database Implementation Environment

Table 10. Application Development Environment

Table 11. Application Module-to-Database Mapping

Table 12. CRUD Operations

Table 13. Business Rules, Validation and Error Handling

Table 14. Reports and Database Operations

Table 15. Test Cases

LIST OF FIGURES

Figure 1. Entity-Relationship Diagram of the Proposed System

Figure 2. Reports and Views Dashboard

Figure 3. Alumni Registration Screen

Figure 4. Student Mentoring Request Screen

Figure 5. Mentor Assignment Screen

Figure 6. Donation Entry Screen

Figure 7. Report Output Dashboard

LIST OF ACRONYMS

Acronym	Expansion
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
ERD	Entity-Relationship Diagram
FK	Foreign Key
GUI	Graphical User Interface
PK	Primary Key
SQL	Structured Query Language
3NF	Third Normal Form

ABSTRACT

Alumni engagement is an important part of a university community, but mentoring requests, alumni records, donations and event information can become difficult to coordinate when maintained separately. A structured information system can bring these activities into a consistent workflow and help staff and students locate relevant records. The objective of this project is to design an Alumni Mentoring & Engagement Management System and demonstrate its main workflows through a browser-based prototype. The proposed logical design uses separate relations for alumni, students, mentoring requests, donations and events, with primary keys, foreign keys and validation rules to maintain data consistency. The supplied simulation provides screens for alumni registration, mentoring-request submission, mentor assignment, donation entry and combined reports for mentor matches, donations and events. Its current implementation stores records in browser local storage and uses JavaScript validation; it does not yet connect to a relational DBMS. Therefore, screenshots are treated as evidence of prototype behaviour, while the relational schema and SQL in this report describe the database implementation plan required for a complete DBMS application. The project demonstrates how database modelling and application workflows can support alumni mentoring and engagement, and identifies the database integration work needed for a production-ready version.

Keywords: Alumni Engagement, Mentoring Management, Relational Database, ER Diagram, SQL, CRUD Operations

CHAPTER 1
PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

Universities maintain relationships with graduates for mentoring, networking, events and contributions. When alumni profiles, student mentoring requests, mentor assignments, donations and event details are tracked independently, it becomes harder to identify suitable mentors, follow pending requests, check contribution records and prepare consolidated summaries. Repeated data entry can also introduce inconsistent email addresses, missing links between records and unreliable reports.

The Alumni Mentoring & Engagement Management System addresses this coordination problem by defining a common data model and a simple set of user workflows. The supplied web simulation demonstrates the main screens and report views. Since the supplied HTML uses browser local storage rather than a live SQL database, this report separates the demonstrated prototype from the proposed relational database implementation.

1.2 Objectives

- Design an ER model and relational schema for alumni, students, mentoring requests, donations and events.
- Define primary keys, foreign keys, uniqueness rules and domain checks to reduce invalid records.
- Prepare a normalized logical design up to Third Normal Form (3NF).
- Demonstrate alumni registration, mentoring-request submission, mentor assignment and donation entry through the supplied interface.
- Specify SQL queries and test cases for database-backed reports and core data operations.

1.3 Scope and Assumptions

Item	Description
In Scope	Alumni profile management; student mentoring requests; assigning an alumni mentor; recording donations; event listing; summary reports; relational schema and SQL design.
Out of Scope / Exclusions	Authentication and role permissions; payment gateway; email/SMS notifications; production deployment; analytics beyond the listed summaries; real institutional data integration.

Assumptions	Each alumni record has a unique email. Each mentoring request is raised by one student and may have at most one active assigned mentor. Donation amount must be greater than zero. Event capacity is a non-negative integer. Sample records are illustrative.

1.4 Proposed Solution Strategy

Solution Element	Project-Specific Description
Primary Users / Roles	Students submit mentoring requests; alumni may act as mentors and donors; a coordinator reviews requests and maintains engagement records.
Major Application Modules	Reports & Views, Register Alumni, Mentoring Request, Assign Mentor and Donations.
Proposed Workflow	Register alumni; capture a student request; select a pending request and alumni mentor; record donations; view mentoring, donation and event summaries.
Important Business Rules	Unique alumni email; one mentor per request at a time; only pending requests can be assigned; donation amount > 0; foreign keys must refer to existing records.
Expected Reports / Outputs	Mentor-match/request list, donation summary and event/capacity list.

1.5 Functional Requirements

Req. ID	Functional Requirement	Priority
FR-01	Register alumni with name, unique email, graduation year and programme.	High
FR-02	Create a mentoring request for a student and topic.	High
FR-03	Assign an alumni mentor to a pending request and update its status.	High
FR-04	Record a donation linked to an alumni record, with a positive amount and date.	High
FR-05	Display joined mentoring, donation and event summaries.	High
FR-06	Persist records and enforce constraints through a relational DBMS in the complete implementation.	High; pending integration

1.6 Entity-Relationship Diagram (ERD)

Figure 1 presents the proposed ER model. A student can submit multiple mentoring requests. Each request belongs to one student and may have zero or one assigned alumni mentor. An alumni member may be linked to multiple mentoring requests and donations. Events are represented as an independent entity because the supplied prototype lists events but does not yet implement event registration.

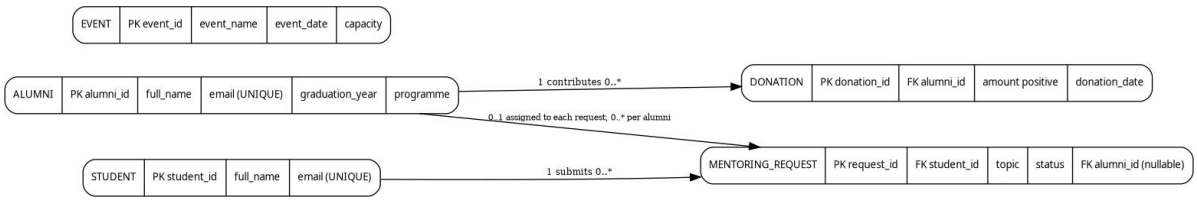


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Entity / Relationship	Major Attributes / Cardinality	Purpose / Rule
ALUMNI	alumni_id (PK), full_name, email (UNIQUE), graduation_year, programme	Stores graduate profile and mentor/donor identity.
STUDENT	student_id (PK), full_name, email (UNIQUE)	Stores student details associated with requests.
MENTORING_REQUEST	request_id (PK), student_id (FK), topic, status, alumni_id (nullable FK)	Each request belongs to one student; an alumni mentor is optional until assignment.
DONATION	donation_id (PK), alumni_id (FK), amount, donation_date	Each donation is linked to one alumni record; amount must be positive.
EVENT	event_id (PK), event_name, event_date, capacity	Stores event details for summary display.
STUDENT–REQUEST	1 : many	One student may create multiple requests; each request has one requesting student.
ALUMNI–DONATION	1 : many	One alumni member may have multiple donation records.

CHAPTER 2
LOGICAL DATABASE DESIGN

2.1 Relational Schema

The following schema is the proposed relational design for implementation in MySQL. PK denotes a primary key and FK denotes a foreign key. The supplied HTML simulation currently uses equivalent JavaScript objects but does not create these SQL tables.

S.No.	Table Name	Purpose
1	ALUMNI(alumni_id PK, full_name, email UNIQUE, graduation_year, programme)	Graduate profile, mentor and donor identity.
2	STUDENT(student_id PK, full_name, email UNIQUE)	Student profile.
3	MENTORING_REQUEST(request_id PK, student_id FK, topic, status, alumni_id FK NULL)	Tracks requests and mentor assignment.
4	DONATION(donation_id PK, alumni_id FK, amount, donation_date)	Stores donation transactions.
5	EVENT(event_id PK, event_name, event_date, capacity)	Stores event and capacity information.

2.2 Keys and Integrity Constraints

Table Name	Key / Constraint	Attribute(s)	Business Rule Enforced
ALUMNI	PRIMARY KEY	alumni_id	Uniquely identifies an alumni record.
ALUMNI	UNIQUE, NOT NULL	email	Prevents duplicate or missing email addresses.
ALUMNI	NOT NULL / CHECK	full_name; graduation_year	Name is required; graduation year must be a plausible positive year.
STUDENT	PRIMARY KEY; UNIQUE	student_id; email	Identifies students and avoids duplicate email addresses.
MENTORING_REQUEST	PRIMARY KEY / FOREIGN KEY	request_id; student_id	Each request has a unique ID and valid student.
MENTORING_REQUEST	FOREIGN KEY, nullable	alumni_id	Assigned mentor must exist; NULL means unassigned.
MENTORING_REQUEST	CHECK	status	Status is limited to Pending, Assigned or Closed.
DONATION	PRIMARY KEY / FOREIGN KEY	donation_id; alumni_id	Each donation is unique and belongs to an existing alumnus.
DONATION	CHECK	amount	Donation amount must be greater than zero.
EVENT	PRIMARY KEY / CHECK / DEFAULT	event_id; capacity	Capacity is non-negative; capacity may default to zero if not supplied.

2.3 Functional Dependencies

- ALUMNI: alumni_id → full_name, email, graduation_year, programme; email → alumni_id, full_name, graduation_year, programme (email is unique).
- STUDENT: student_id → full_name, email; email → student_id, full_name (email is unique).

- MENTORING_REQUEST: request_id → student_id, topic, status, alumni_id.
- DONATION: donation_id → alumni_id, amount, donation_date.
- EVENT: event_id → event_name, event_date, capacity.

2.4 Normalization up to Third Normal Form (3NF)

The design separates alumni, students, requests, donations and events instead of repeating their descriptive details in one large table. Each relation uses a single-valued attribute set and a primary key. Non-key attributes depend on the whole key, and descriptive attributes are stored with their own entity rather than copied into transaction rows. On these assumptions, the proposed relations satisfy 1NF, 2NF and 3NF. BCNF is also expected for the listed dependencies because each stated determinant is a candidate key; this should be rechecked if additional dependencies are introduced.

Relation Name	1NF	2NF	3NF	BCNF / Remarks
ALUMNI	Yes	Yes	Yes	Expected for stated dependencies
STUDENT	Yes	Yes	Yes	Expected for stated dependencies
MENTORING_REQUEST	Yes	Yes	Yes	Expected for stated dependencies
DONATION	Yes	Yes	Yes	Expected for stated dependencies
EVENT	Yes	Yes	Yes	Expected for stated dependencies

2.5 Data Dictionary

Table	Attribute	Data Type	Key / Constraint	Description
ALUMNI	alumni_id	INT	PK, AUTO_INCREMENT	Unique alumni identifier.
ALUMNI	full_name	VARCHAR(100)	NOT NULL	Alumni full name.
ALUMNI	email	VARCHAR(150)	UNIQUE, NOT NULL	Contact email.
ALUMNI	graduation_year	SMALLINT	NOT NULL, CHECK	Year of graduation.
ALUMNI	programme	VARCHAR(100)	NOT NULL	Academic programme.
STUDENT	student_id	INT	PK, AUTO_INCREMENT	Unique student identifier.
STUDENT	full_name	VARCHAR(100)	NOT NULL	Student name.
STUDENT	email	VARCHAR(150)	UNIQUE, NOT NULL	Student email.
MENTORING_REQUEST	request_id	INT	PK, AUTO_INCREMENT	Unique request identifier.
MENTORING_REQUEST	student_id	INT	FK, NOT NULL	Student who submitted the request.
MENTORING_REQUEST	topic	VARCHAR(200)	NOT NULL	Requested mentoring area.
MENTORING_REQUEST	status	VARCHAR(20)	DEFAULT Pending, CHECK	Request workflow status.
MENTORING_REQUEST	alumni_id	INT	FK, NULL	Assigned alumni mentor, if any.
DONATION	donation_id	INT	PK, AUTO_INCREMENT	Unique donation identifier.
DONATION	alumni_id	INT	FK, NOT NULL	Alumni donor.
DONATION	amount	DECIMAL(10,2)	CHECK > 0	Donation amount.
DONATION	donation_date	DATE	NOT NULL	Date donation was recorded.
EVENT	event_id	INT	PK, AUTO_INCREMENT	Unique event identifier.
EVENT	event_name	VARCHAR(150)	NOT NULL	Event title.
EVENT	event_date	DATE	NOT NULL	Scheduled date.
EVENT	capacity	INT	DEFAULT 0, CHECK >= 0	Maximum event capacity.

2.6 Implementation of Logical Design: DDL and Sample Data

Item	Details
DBMS	MySQL is the proposed relational DBMS for the complete implementation.
DBMS Version	Not specified in the supplied files; record the installed version when implemented.
Number of Tables	Five proposed relations: ALUMNI, STUDENT, MENTORING_REQUEST, DONATION and EVENT.
Sample Data	The supplied simulation initializes two alumni, two students, two mentoring requests, two donations and two events in browser local storage. These are prototype records, not SQL table rows.

Representative DDL for the core entities is shown below. The complete SQL script is included in Appendix A.

```
CREATE TABLE Alumni (
  alumni_id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  graduation_year SMALLINT NOT NULL CHECK (graduation_year > 0),
  programme VARCHAR(100) NOT NULL
);

CREATE TABLE Student (
  student_id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE
);

CREATE TABLE Mentoring_Request (
  request_id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT NOT NULL, topic VARCHAR(200) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'Pending',
  alumni_id INT NULL,
  CHECK (status IN ('Pending','Assigned','Closed')),
  FOREIGN KEY (student_id) REFERENCES Student(student_id),
  FOREIGN KEY (alumni_id) REFERENCES Alumni(alumni_id)
);
```

The implementation should also create Donation with a CHECK (amount > 0) and an FK to Alumni, and Event with a non-negative capacity constraint. The simulation's positive-amount and unique-email checks are currently implemented in JavaScript; database-level enforcement remains pending.

CHAPTER 3
APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The supplied application is a browser-based prototype titled "Alumni Mentoring & Engagement Management System." Its navigation contains five modules: Reports & Views, Register Alumni, Mentoring Request, Assign Mentor (3NF) and Donations. It initializes illustrative records and stores changes in browser local storage. The interface demonstrates the intended workflow and client-side validation, but it is not yet connected to MySQL or another relational DBMS. A database connector, server-side operations and persistence tests are required before it can be described as a fully database-connected application.

3.2 Development Environment and Database Connectivity

Item	Details
Programming Language	HTML, CSS and JavaScript for the supplied simulation; no Java/Python source was supplied.
DBMS	No live DBMS connection in the current prototype. MySQL is proposed for the relational implementation.
IDE / Framework / GUI Toolkit	Browser-based interface using plain HTML, CSS and JavaScript; no framework identified.
Database Connector / Driver	None present in the supplied HTML. A MySQL connector/driver and backend API are future implementation requirements.
Operating Environment	Web browser; screenshots captured from the supplied local HTML simulation.
Repository Link	Not supplied.

3.3 Application Modules and Database Mapping

Module / Screen	Purpose	Tables Used (proposed)	Main Operations
Register Alumni	Collect graduate profile fields.	ALUMNI	Create; check unique email.
Mentoring Request	Submit a topic for a student.	STUDENT, MENTORING_REQUEST	Create student if needed; create request.
Assign Mentor	Assign an alumnus to a pending request.	MENTORING_REQUEST, ALUMNI	Read pending requests and alumni; update request.
Donations	Record a contribution.	DONATION, ALUMNI	Read alumni; create donation; validate amount.
Reports & Views	Display mentoring matches, donations and events.	MENTORING_REQUEST, STUDENT, ALUMNI, DONATION, EVENT	Read/join and summarize records.

3.4 CRUD Operations

Operation	Application Feature / Screen	Table(s) Affected	Evidence / Current Status
Create	Register Alumni; Mentoring Request; Donations	ALUMNI; STUDENT and MENTORING_REQUEST; DONATION	Screens and client-side handlers are present; writes go to local storage.
Read / Search	Reports & Views; dropdowns	All proposed relations	Displays seeded and newly

			entered local records; no SQL query is executed.
Update	Assign Mentor	MENTORING_REQUEST	Changes status and alumni ID in local storage.
Delete / Controlled Removal	No delete screen or handler is present.	Not implemented	Deletion should be added only with confirmation and referential-integrity handling.

3.5 Business Rules, Validation and Error Handling

Rule / Validation	How Implemented in Prototype	Evidence / Limitation
Unique alumni email	JavaScript checks existing alumni email before insertion.	Duplicate triggers an error message; not enforced by a DBMS constraint.
Required fields and email format	HTML required attributes and email input type.	Browser-level validation; server-side validation is absent.
Positive donation amount	Input min=0.01 and JavaScript check amount <= 0.	Client-side rule only; SQL CHECK not yet active.
One mentor per request	Assignment operates on a selected pending request and sets one alumni ID.	The workflow replaces the assigned ID; no database transaction or concurrency control.
Error handling	Success/error messages are displayed in the page.	No SQL exceptions, parameterized queries or rollback are present because no DB connector exists.
Referential integrity	Dropdowns are populated from local alumni/request objects.	Logical links exist in JavaScript; relational foreign keys remain to be implemented.

3.6 Reports and Database Operations

Report / Operation	SQL / Database Concept Used in Complete Version	Application Output
Mentor Matches & Requests	JOIN Student to Mentoring_Request and LEFT JOIN Alumni on assigned mentor.	Request ID, student, topic, status and assigned mentor.
Alumni Donations Summary	JOIN Donation to Alumni; order by donation date.	Donation ID, alumni name, amount and date.
Events & Capacity Status	SELECT event_name, event_date, capacity FROM Event.	Event name, date and capacity.
Pending Request Dropdown	SELECT requests WHERE status = Pending; SELECT alumni.	Options used for mentor assignment.
Constraint checks	UNIQUE email, FK references, CHECK amount > 0 and status domain.	Prevents invalid data in the database-backed version.

3.7 Application Screens

Figures 2–7 show the supplied simulation. The first five screens capture the core modules. Figure 7 repeats the report dashboard as the report/output evidence requested by the template. The donation workflow itself is shown separately in Figure 6.

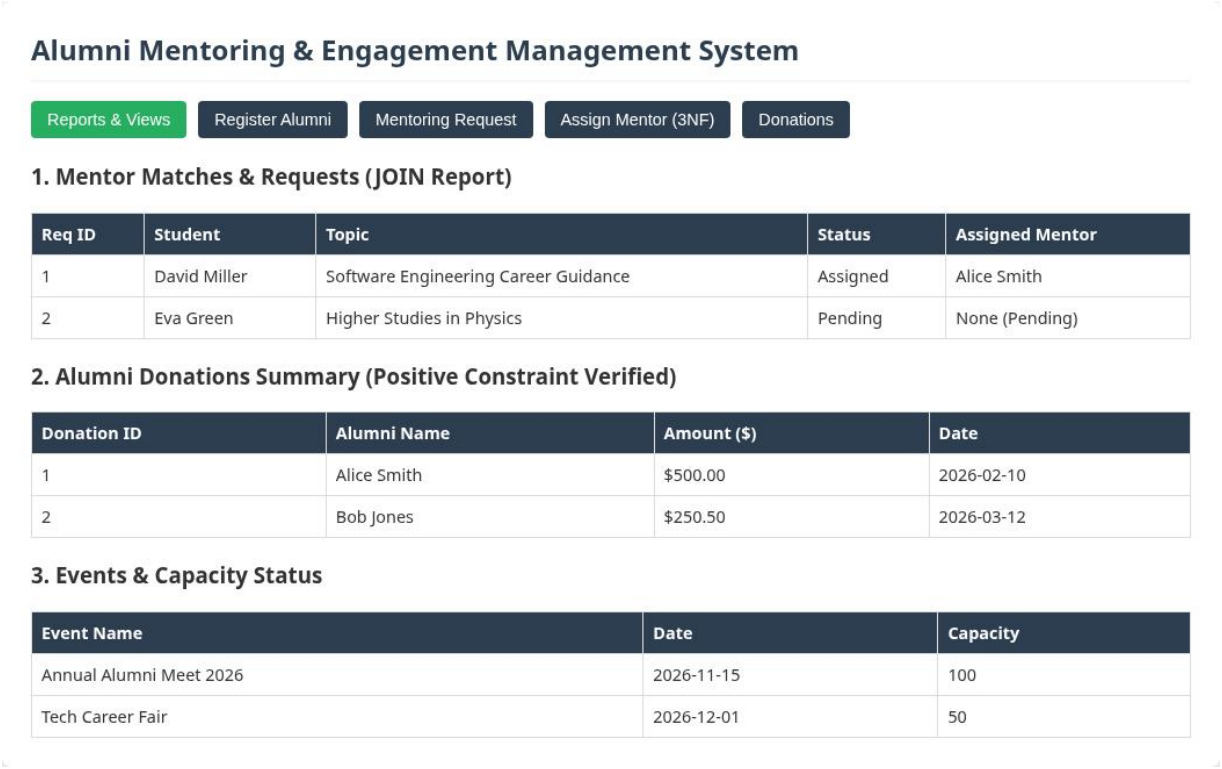


Figure 2. Reports and Views Dashboard

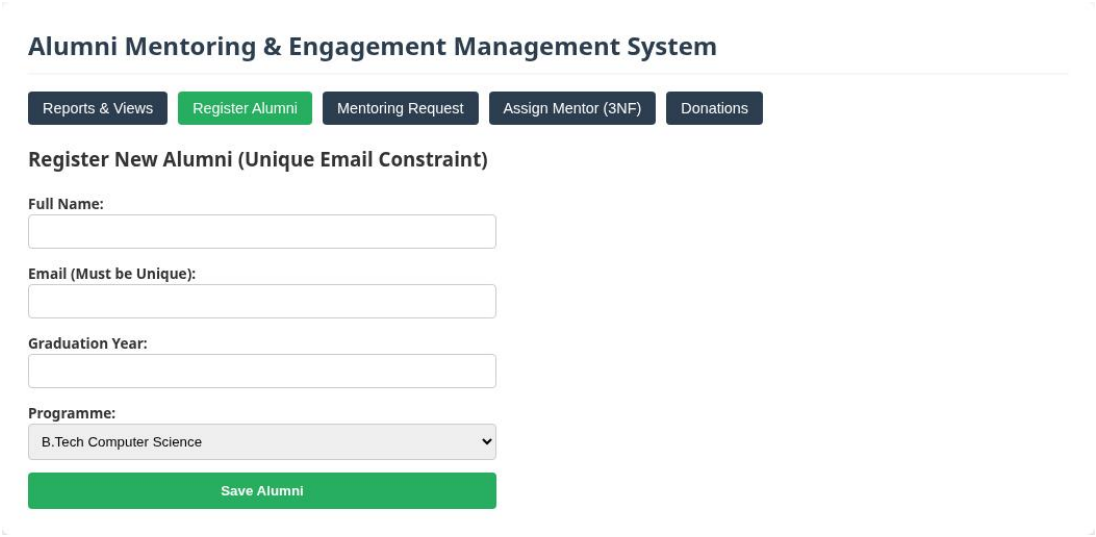


Figure 3. Alumni Registration Screen

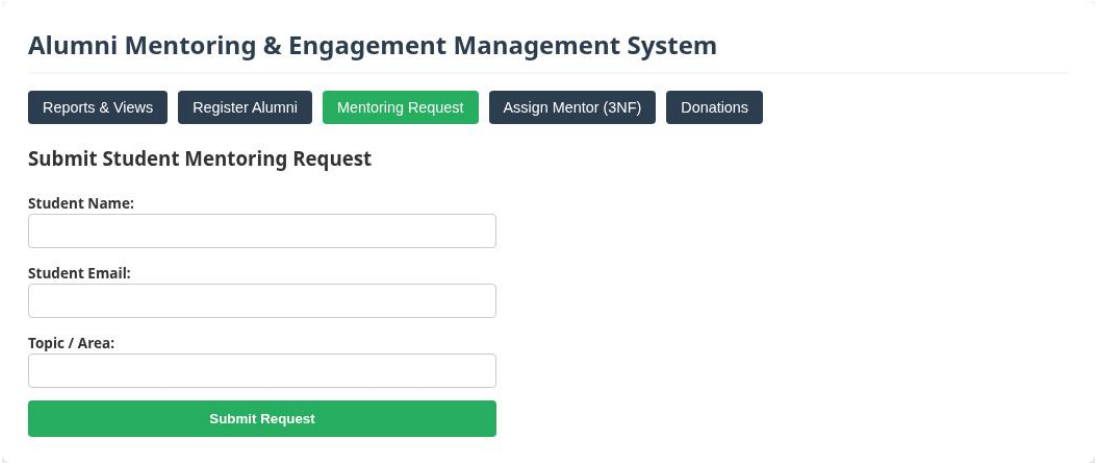


Figure 4. Student Mentoring Request Screen

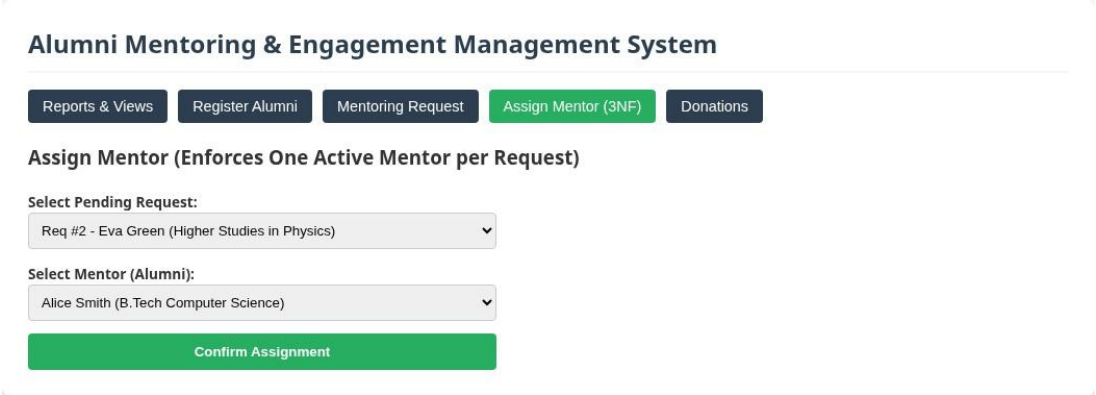


Figure 5. Mentor Assignment Screen

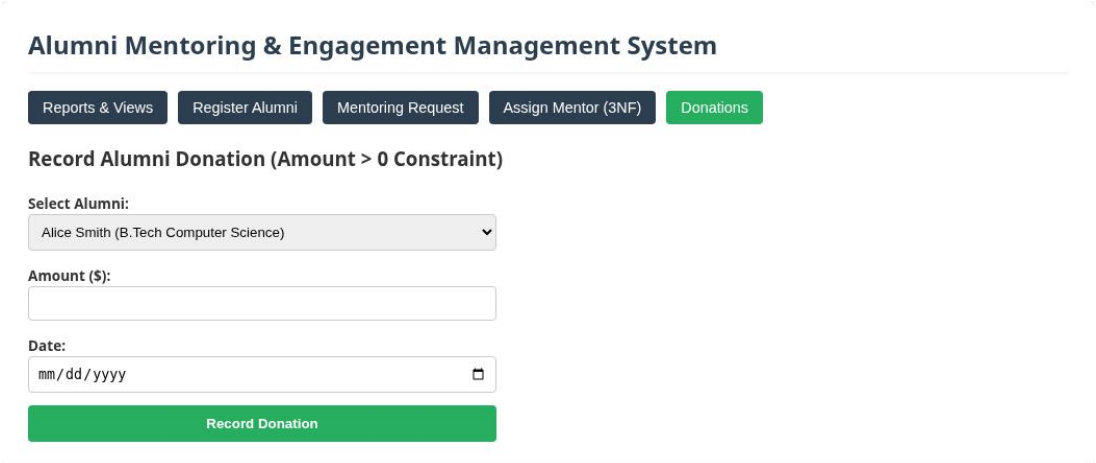


Figure 6. Donation Entry Screen

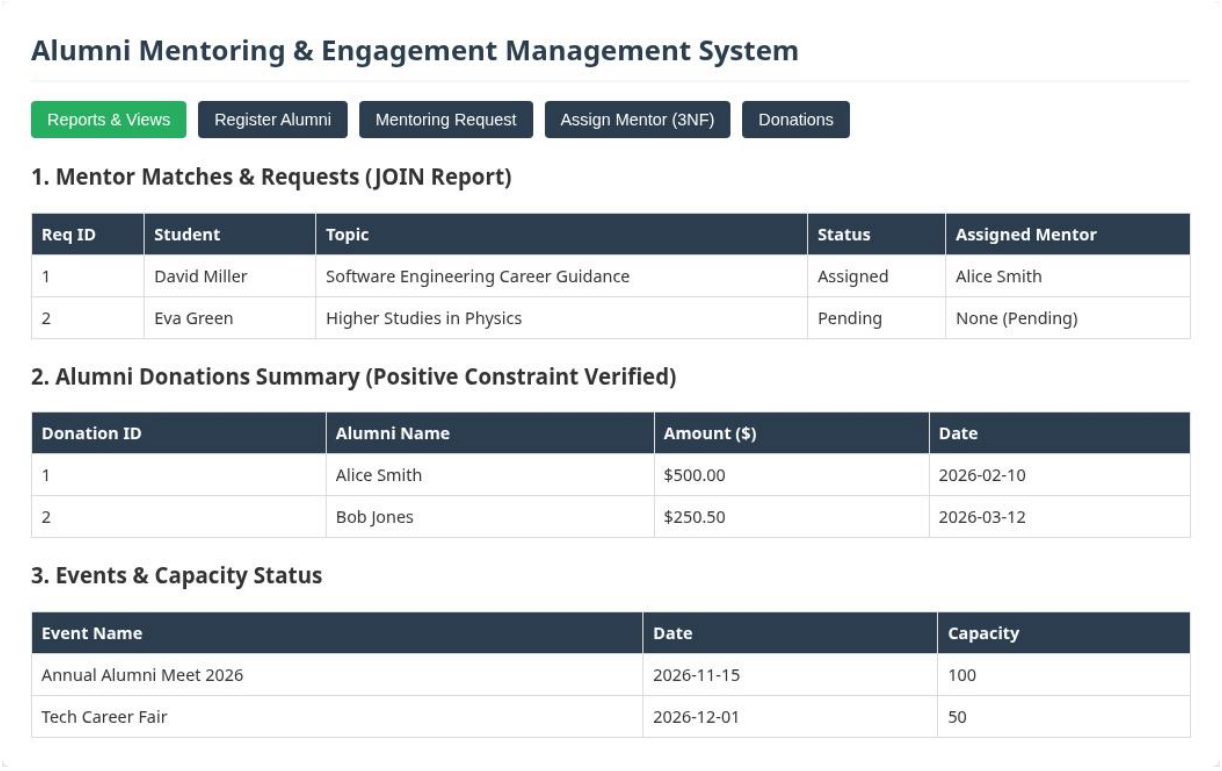


Figure 7. Report Output Dashboard

3.8 Testing and Results

The results below describe the supplied simulation and a limited set of interactions visible in its code. The positive transaction screenshot verifies that the interface can add a donation to local storage and refresh its report. Negative cases described from code paths are marked as code-inspected rather than database-tested.

Test ID	Feature	Input / Action	Expected Result	Actual Result	Status
TC-01	Reports	Open Reports & Views on initial load.	Show mentoring, donations and events.	Seeded rows are rendered by refreshUI().	Pass – observed
TC-02	Alumni registration	Enter a new unique email and submit.	Record is added and success message appears.	Handler adds record to local storage and refreshes UI.	Pass – code path; UI form present
TC-03	Duplicate email	Register an email already in alumni list.	Reject duplicate and show error.	Code checks email and calls error message.	Pass – code-inspected
TC-04	Mentoring request	Submit valid student name, email and topic.	Create student if needed and a Pending request.	Handler creates local object(s) and refreshes UI.	Pass – code-inspected
TC-05	Assign mentor	Choose pending request and alumni, confirm.	Request becomes Assigned with mentor name.	Handler updates local request status and alumnId.	Pass – code-inspected
TC-06	Donation validation	Attempt zero/negative donation amount.	Reject invalid amount.	Input min and JavaScript check are present.	Pass – code-inspected; not DB-tested
TC-07	Positive donation	Enter a positive amount and valid date, then submit.	Add donation and show it in summary.	Code handler appends the donation to local storage and refreshes the UI; this interaction was not separately captured.	Pass – code-inspected

Key results: the simulation demonstrates the navigation and client-side workflows for registering alumni, submitting mentoring requests, assigning a mentor, recording a donation and displaying three summaries. Its JavaScript contains duplicate-email and positive-donation checks. However, it does not contain SQL statements, a database connector, server-side validation, database transactions or a delete workflow. Consequently, the application/database integration requirement is only partially satisfied by the current supplied artifact; completing the MySQL connection and testing the same cases against actual tables is the most important next step.

CHAPTER 4
CONCLUSION

The Alumni Mentoring & Engagement Management System models a practical university engagement scenario involving alumni profiles, student mentoring requests, mentor assignment, donations and events. The report defines the system scope, functional requirements, ER model, relational schema, keys, integrity constraints, functional dependencies and normalization up to 3NF. The proposed design separates master data from requests and transactions to reduce duplication and support consistent reports.

The supplied browser prototype demonstrates the principal screens and local workflows, including validation messages and joined-style summaries. Its current data storage is browser local storage, not a relational DBMS. The project therefore provides a useful interface prototype and database design, while a complete DBMS deliverable still requires SQL table creation, a backend connection, parameterized database operations and integration testing against the implemented schema.

4.1 Limitations and Future Enhancements

- Connect the interface to MySQL through a backend service and enforce primary keys, foreign keys, UNIQUE and CHECK constraints in the database.
- Add authentication, role-based access, audit logs, secure input handling and server-side validation.
- Implement delete/archive operations with confirmation, transactional consistency and referential-integrity rules.
- Add event registration and capacity tracking, exportable reports, notifications and deployment documentation.

5. REFERENCES

[1] Oracle, “MySQL 8.4 Reference Manual,” Oracle. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/. Accessed: Oct. 12, 2026.

[2] Oracle, “MySQL Tutorial,” Oracle. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/tutorial.html. Accessed: Oct. 12, 2026.

[3] WHATWG, “HTML Living Standard,” [Online]. Available: https://html.spec.whatwg.org/. Accessed: Oct. 12, 2026.

[4] MDN Web Docs, “Window: localStorage property,” Mozilla. [Online]. Available: https://developer.mozilla.org/en-US/docs/Web/API/Window/localStorage. Accessed: Oct. 12, 2026.

The supplied HTML simulation and the provided report template were used as project materials. References above support the technical concepts; the report does not claim that a live MySQL connection exists in the supplied prototype.

6. APPENDICES

APPENDIX A – COMPLETE DATABASE SCRIPT

```
CREATE DATABASE IF NOT EXISTS alumni_engagement;
USE alumni_engagement;

CREATE TABLE Alumni (
  alumni_id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE,
  graduation_year SMALLINT NOT NULL CHECK (graduation_year > 0),
  programme VARCHAR(100) NOT NULL
);

CREATE TABLE Student (
  student_id INT AUTO_INCREMENT PRIMARY KEY,
  full_name VARCHAR(100) NOT NULL,
  email VARCHAR(150) NOT NULL UNIQUE
);

CREATE TABLE Mentoring_Request (
  request_id INT AUTO_INCREMENT PRIMARY KEY,
  student_id INT NOT NULL,
  topic VARCHAR(200) NOT NULL,
  status VARCHAR(20) NOT NULL DEFAULT 'Pending',
  alumni_id INT NULL,
  CONSTRAINT chk_request_status CHECK (status IN ('Pending','Assigned','Closed')),
  CONSTRAINT fk_request_student FOREIGN KEY (student_id) REFERENCES Student(student_id),
  CONSTRAINT fk_request_alumni FOREIGN KEY (alumni_id) REFERENCES Alumni(alumni_id)
);

CREATE TABLE Donation (
  donation_id INT AUTO_INCREMENT PRIMARY KEY,
  alumni_id INT NOT NULL,
  amount DECIMAL(10,2) NOT NULL CHECK (amount > 0),
  donation_date DATE NOT NULL,
  CONSTRAINT fk_donation_alumni FOREIGN KEY (alumni_id) REFERENCES Alumni(alumni_id)
);

CREATE TABLE Event (
  event_id INT AUTO_INCREMENT PRIMARY KEY,
  event_name VARCHAR(150) NOT NULL,
  event_date DATE NOT NULL,
  capacity INT NOT NULL DEFAULT 0 CHECK (capacity >= 0)
);

-- Illustrative sample records
INSERT INTO Alumni(full_name,email,graduation_year,programme) VALUES
('Alice Smith','alice.smith@alumni.edu',2020,'B.Tech Computer Science'),
('Bob Jones','bob.jones@alumni.edu',2018,'B.Sc Physics');
INSERT INTO Student(full_name,email) VALUES
('David Miller','david.m@student.edu'),('Eva Green','eva.g@student.edu');
INSERT INTO Mentoring_Request(student_id,topic,status,alumni_id) VALUES
(1,'Software Engineering Career Guidance','Assigned',1),
(2,'Higher Studies in Physics','Pending',NULL);
INSERT INTO Donation(alumni_id,amount,donation_date) VALUES
(1,500.00,'2026-02-10'),(2,250.50,'2026-03-12');
INSERT INTO Event(event_name,event_date,capacity) VALUES
('Annual Alumni Meet 2026','2026-11-15',100),
('Tech Career Fair','2026-12-01',50);

-- Mentor match report
SELECT r.request_id, s.full_name AS student, r.topic, r.status,
       a.full_name AS assigned_mentor
FROM Mentoring_Request r
JOIN Student s ON s.student_id = r.student_id
LEFT JOIN Alumni a ON a.alumni_id = r.alumni_id;

-- Donations summary
SELECT d.donation_id, a.full_name AS alumni_name, d.amount, d.donation_date
FROM Donation d JOIN Alumni a ON a.alumni_id = d.alumni_id
ORDER BY d.donation_date;

-- Event and capacity list
SELECT event_name, event_date, capacity FROM Event ORDER BY event_date;
```

APPENDIX B – APPLICATION SOURCE CODE

Application source supplied: _init_.html (Alumni Mentoring & Engagement Management System). It contains the HTML/CSS interface and JavaScript localStorage handlers. No repository URL or separate Java/Python source folder was supplied.

APPENDIX C – ADDITIONAL TEST DATA AND OUTPUTS

The initial simulation data consists of alumni Alice Smith and Bob Jones; students David Miller and Eva Green; one assigned and one pending mentoring request; two donation records; and two event records. Figure 7 is the supplied prototype's report dashboard. The donation form and handler provide the intended entry workflow, but a post-submission output screenshot was not separately captured.